Time taken	2 days 6 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int FIRSTZERO(int arr[], int low, int high) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
            return mid;
        }
        if (arr[mid] == 1) {
```

```
            return FIRSTZERO(arr, mid + 1, high);
        }
        return FIRSTZERO(arr, low, mid - 1);
    }

    return -1;
}
int countZeros(int arr[], int n) {
    int first = FIRSTZERO(arr, 0, n - 1);
    if (first == -1) {
        return 0;
    }

    return (n - first);
}
int main() {
    int m;
    if (scanf("%d", &m) != 1 || m <= 0) {
        return 0;
    }

    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    printf("%d\n", countZeros(arr, m));

    return 0;
}
```

	Input	Expected	Got	

✔	5 1 1 1 0 0	2	2	✔

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int countInRange(int nums[], int target, int low, int high) {
    int count = 0;
    for(int i=low;i<=high;i++){
        if(nums[i]==target){
            count++;
        }
    }
    return count;
}
int majorityElementRec(int nums[], int low, int high){
    if(low==high){
        return nums[low];
    }
    int mid = low + (high - low)/2;
    int leftMajority = majorityElementRec(nums, low, mid);
    int rightMajority = majorityElementRec(nums, mid+1, high);
```

```
    int rightMajority = majorityElementRec(nums, mid+1, high);

    if(leftMajority == rightMajority){
        return leftMajority;
    }
    int leftCount = countInRange(nums, leftMajority, low, high);
    int rightCount = countInRange(nums, rightMajority, low, high);

    return leftCount > rightCount ? leftMajority : rightMajority;
}
int majorityElement(int nums[], int n) {
    return majorityElementRec(nums, 0, n - 1);
}

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        int nums[n];
        for (int i = 0; i < n; i++) {
            scanf("%d", &nums[i]);
        }
        printf("%d\n", majorityElement(nums, n));
    }
    return 0;
}
```

	Input	Expected	Got	
✔		3	3	✔

	3 3 2 3			

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Started on	Wednesday, 30 September 2026, 9:56 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 10:05 PM
Time taken	9 mins 10 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
#include<stdlib.h>
int findFloor(int arr[], int low, int high, int x){
    int floor_val = -1;
    while(low<=high){
        int mid = low + (high - low)/2;
        if(arr[mid] == x){
            return arr[mid];
        }
        else if(arr[mid]<x){
            floor_val = arr[mid];
            low = mid + 1;
        }
        else{
            high = mid - 1;
        }
    }
    return floor_val;
}
int main(){
    int n,x;
    if(scanf("%d", &n) != 1) return 0;
    int *arr = (int *)malloc(n*sizeof(int));
    for(int i = 0;i<n;i++){
        if(scanf("%d",&arr[i])!=1) return 0;
    }
    if(scanf("%d",&x)!=1) return 0;
```

```
28      int result = findFloor(arr, 0, n-1, x);
29      printf("%d\n", result);
30
31      free(arr);
32      return 0;
33  }
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Wednesday, 30 September 2026, 10:07 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 10:20 PM
Time taken	13 mins 39 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
#include<stdlib.h>
void findSumPair(int arr[], int low, int high, int x){
    if(low>=high){
        printf("No\n");
        return;
    }
    int current_sum = arr[low]+arr[high];
    if(current_sum == x){
        printf("%d\n", arr[low]);
        printf("%d\n", arr[high]);
        return;
    }
    else if(current_sum < x){
        findSumPair(arr, low+1, high,x);
    }
    else{
        findSumPair(arr, low, high-1, x);
    }
}
int main(){
    int n,x;
    if(scanf("%d",&n)!=1) return 0;
    int *arr = (int *)malloc(n*sizeof(int));
    for(int i=0;i<n;i++){
        if(scanf("%d",&arr[i])!=1) return 0;
    }
    if(scanf("%d",&x)!=1) return 0;
```

```
    findSumPair(arr,0,n-1,x);
    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Wednesday, 30 September 2026, 10:21 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 10:32 PM
Time taken	10 mins 53 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```
#include<stdio.h>
#include<stdlib.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = (low-1);
    for(int j=low;j<high;j++){
        if(arr[j]<pivot){
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i+1], &arr[high]);
    return (i+1);
}
void quickSort(int arr[], int low, int high){
    if(low<high){
        int pi=partition(arr,low,high);
        quickSort(arr,low,pi-1);
```

```
        quickSort(arr,pi+1,high);
    }
}
int main(){
    int n;
    if(scanf("%d",&n)!=1) return 0;
    int *arr = (int *)malloc(n*sizeof(int));
    for(int i =0;i<n;i++){
        if(scanf("%d",&arr[i])!=1) return 0;
    }
    quickSort(arr,0,n-1);
    for(int i = 0;i<n;i++){
        printf("%d",arr[i]);
        if(i<n-1){
            printf(" ");
        }
    }
    printf("\n");
    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.